

Programação Orientada a Objetos - CST em Análise e Desenvolvimento de Sistemas

# Herança

Prof. Emerson Ribeiro de Mello

[mello@ifsc.edu.br](mailto:mello@ifsc.edu.br)



**INSTITUTO FEDERAL**  
Santa Catarina  
Campus São José

# Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

# Herança

## Genética

Processo pelo qual um organismo transmite características físicas para seus descendentes.

# Herança

## Genética

Processo pelo qual um organismo transmite características físicas para seus descendentes.

## Programação Orientada a Objetos

Permite que uma classe (subclasse) reutilize e estenda atributos e métodos de outra classe (superclasse).

# Herança

- **Herança** permite criar novas classes a partir de classes já existentes, facilitando o desenvolvimento de sistemas complexos
- A **subclasse** (ou classe filha) herda atributos e métodos da **superclasse** (ou classe pai)
- A subclasse pode adicionar ou modificar funcionalidades da superclasse

# Herança

- **Herança** permite criar novas classes a partir de classes já existentes, facilitando o desenvolvimento de sistemas complexos
- A **subclasse** (ou classe filha) herda atributos e métodos da **superclasse** (ou classe pai)
- A subclasse pode adicionar ou modificar funcionalidades da superclasse

## Quando usar herança?

Use herança quando diferentes classes precisam compartilhar características e comportamentos comuns, mas também possuem particularidades

## Exemplo: Sistema para cadastro de produtos

- Uma indústria da área de telecomunicações necessita de um sistema para cadastrar os produtos que fabrica
  - **Aparelho telefônico**
- As informações necessárias para o cadastro são:
  - código, número de série, modelo, cor, peso, dimensões (AxLxP)

## Exemplo: Sistema para cadastro de produtos

- Uma indústria da área de telecomunicações necessita de um sistema para cadastrar os produtos que fabrica
  - **Aparelho telefônico**
- As informações necessárias para o cadastro são:
  - código, número de série, modelo, cor, peso, dimensões (AxLxP)

| Telefone                                                                                     |
|----------------------------------------------------------------------------------------------|
| -codigo: int<br>-numSerie: String<br>-modelo: String<br>-peso: double<br>-dimensao: Dimensao |
| +Telefone(c: int, n: String, m: String, p: double, d: Dimensao)<br>+toString(): String       |

## Exemplo: Sistema para cadastro de produtos

- A empresa começou a fabricar também **telefones sem fio**
- Os **telefones sem fio** compartilham todas as características de um **telefone**, porém possuem novas características
  - frequência, quantidade de canais, distância de operação
- O atual sistema não permite cadastrar essas novas informações

## Exemplo: Sistema para cadastro de produtos

- A empresa começou a fabricar também **telefones sem fio**
- Os **telefones sem fio** compartilham todas as características de um **telefone**, porém possuem novas características
  - frequência, quantidade de canais, distância de operação
- O atual sistema não permite cadastrar essas novas informações

### O que fazer?

- 1 Criar uma nova classe para telefone sem fio, duplicando tudo da classe telefone e adicionando as novas características?
- 2 Herdar as características comuns da classe telefone e adicionar as particulares do telefone sem fio?

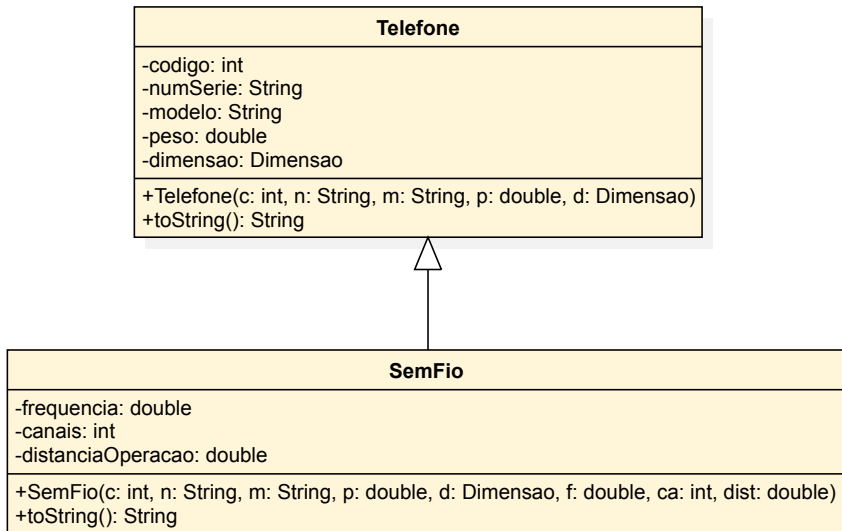
## Exemplo: Sistema para cadastro de produtos

- A empresa começou a fabricar também **telefones sem fio**
- Os **telefones sem fio** compartilham todas as características de um **telefone**, porém possuem novas características
  - frequência, quantidade de canais, distância de operação
- O atual sistema não permite cadastrar essas novas informações

### O que fazer?

- 1 Criar uma nova classe para telefone sem fio, duplicando tudo da classe telefone e adicionando as novas características?
- 2 Herdar as características comuns da classe telefone e adicionar as particulares do telefone sem fio?

## Herança da classe Telefone



## Herança: Superclasse Telefone

```
public class Telefone{
    private int codigo;
    private String numSerie;
    private String modelo;
    private double peso;
    private Dimensao dim;

    public Telefone(int c, String s, String m, double p, Dimensao d){
        this.codigo = c; this.peso = p; this.dim = d;
        this.numSerie = s; this.modelo = m;
    }

    public String toString(){
        // TODO melhorar a String gerada
        return "Codigo: " + this.codigo + " Dimensao: " + this.dim.toString();
    }
}
```

# Herança: Subclasse SemFio

```
public class Semfio extends Telefone{
    private double frequencia;
    private int canais;
    private double distancia;

    public SemFio(int c, String s, String m, double p, Dimensao d, double f, int ca, double dist){
        super(c, s, m, p, d); // invocando o construtor da superclasse
        this.frequencia = f;
        this.distancia = dist;
        this.canais = ca;
    }

    // sobrescrita do metodo da superclasse
    @Override
    public String toString(){
        // invocando o metodo de mesmo nome da superclasse
        return super.toString() + " Freq: " + this.frequencia + " Canais: " + this.canais + " Distancia: " + this.
            distancia;
    }
}
```

# Herança: Criando instâncias de Telefone e SemFio

```
public class Principal{

    public static void main(String[] args){

        Telefone t = new Telefone(1,"ABC","MesaTel",0.5, new Dimensao(10,10,5));

        SemFio sf = new SemFio(2,"DEF","LivreTel",0.7, new Dimensao(20,8,7), 2400, 11, 100);

        System.out.println(t);
        System.out.println(sf);
    }
}
```

# Sobrescrita de método

Uma subclasse pode sobrescrever um método da superclasse que tenha a mesma assinatura

```
public class Telefone{
    public void ola(){
        System.out.println("Ola, sou um telefone");
    }
}

public class SemFio extends Telefone{
    public void ola(){
        System.out.println("Ola, sou um telefone sem fio");
    }
}

public class Principal{
    public static void main(String args[]){
        Telefone t = new Telefone();
        SemFio s = new SemFio();
        t.ola(); // Ola, sou um telefone
        s.ola(); // Ola, sou um telefone sem fio
    }
}
```

## Membros públicos, privados e protegidos

- Os membros **privados** de uma classe só podem ser acessados pelos demais membros desta mesma classe
- Os membros **públicos** de uma classe podem ser acessados por qualquer outra classe
- O modificador de acesso **protected** apresenta uma restrição intermediária entre o **private** e o **public**.
- Membros **protegidos** podem ser acessados pelos demais membros da classe, pelas demais classes do pacote e pelas classes derivadas.

## Modificador de acesso protected: exemplo

```
package produtos;

public class Telefone{
    private String marca;
    protected String modelo;
    public float peso;
}
```

```
package produtos;

public class SemFio extends Telefone{
    private float frequencia;
    public void modificador(){
        this.frequencia = 900; // acesso ok
        this.modelo = "ABC"; // acesso ok
        this.peso = 0.5; // acesso ok
        this.marca = "GrandTel"; // erro! Nao permitido
    }
}
```

## Modificador de acesso protected: exemplo

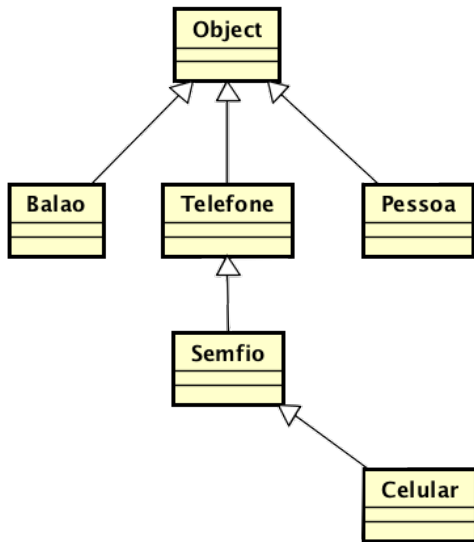
```
package poo;
import produtos.Telefone;
import produtos.SemFio;

public class Principal{
    public static void main(String[] args){
        Telefone t = new Telefone();
        SemFio sf = new SemFio();

        // invocando um membro public
        t.peso = 0.6; // acesso ok
        // invocando um membro protected
        t.modelo = "DEF"; // erro!
        // invocando um membro private
        t.marca = "GT"; // erro!
    }
}
```

# Associação do tipo Herança em Java

- Toda classe herda implicitamente da classe Object
- Associação de herança pode ser lida como **é um**
  - Celular **é um** Telefone
  - Semfio **é um** Telefone
  - Celular **é um** Semfio? **Sim**
  - Semfio **é um** Celular? **Não**
  - Telefone **é um** Celular? **Não**

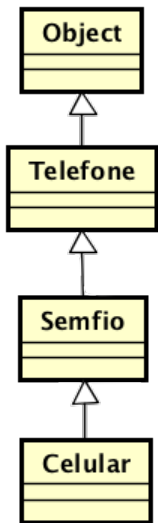


# Exercícios

- Para cada um dos exercícios abaixo, crie um diagrama de classes UML com associações de herança
  - Você pode criar as classes que julgar necessário
  - Cada classe deve ter pelo menos um atributo e um método
- É necessário pensar em um contexto (domínio) para o problema. Exemplos: escola, biblioteca, transporte, zoológico.

- ① Aluno, professor, coordenador, diretor
- ② Livro, revista, jornal, gibi
- ③ Carro, avião, barco, caminhão
- ④ Gato, cachorro, arara, galinha, baleia, tubarão

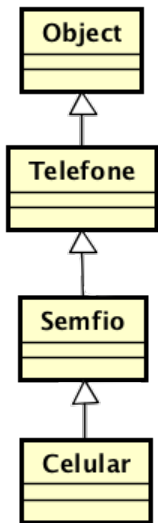
## Coerção de tipos (*typecasting*) – ou conversão de tipos



```
Telefone a = new Telefone();
Semfio b = new SemFio();
Celular c = new Celular();
```

- Celular **é um** Telefone?
- Telefone **é um** Celular?

# Coerção de tipos (*typecasting*) – ou conversão de tipos



```
Telefone a = new Telefone();
Semfio b = new Semfio();
Celular c = new Celular();
```

- Celular **é um** Telefone? **SIM!**
- Telefone **é um** Celular? **Não**

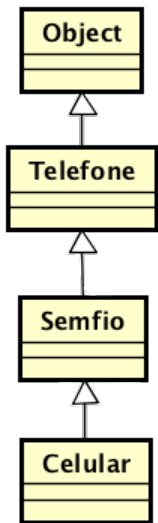
## *typecasting*

O uso do objeto de um tipo na referência de um outro tipo

```
Telefone d = new Celular(); // OK, coerção implícita
Object e = new Semfio(); // OK, coerção implícita
Celular f = (Celular) d; //OK, coerção explícita
```

```
Celular g = a; // ERRO! Telefone não é Celular
Celular h = (Celular) e; // ERRO! Semfio não é Celular
```

# Coerção de tipos (*typecasting*) – ou conversão de tipos



## Operador instanceof

Teste lógico para verificar o tipo de um objeto

```
Telefone produtos[] = new Telefone[3];
produtos[0] = new Telefone();
produtos[1] = new Semfio();
produtos[2] = new Celular();

for(Telefone t : produtos){

    // Se o objeto for uma instância de Celular, faça typecasting
    if (t instanceof Celular c ){

        String sistemaOperacional = c.getSistemaOperacional();
        System.out.println("Celular com SO: " + sistemaOperacional);

        ....
    }
}
```

# Exercícios

# Jogo Java of Empires

Fortemente inspirado no *Age of Empires* da Microsoft – <https://www.ageofempires.com>



Fonte: Microsoft

- **Aldeão**
- Vida: 25
- Ataque: 1
- Velocidade: 0.8



Fonte: Microsoft

- **Arqueiro**
- Vida: 35
- Ataque: 2
- Velocidade: 1



Fonte: Microsoft

- **Cavaleiro**
- Vida: 50
- Ataque: 3
- Velocidade: 2

# Jogo Java of Empires

## Herança de classes

- **Modelagem UML**

- Todos os personagens devem ser capazes de se mover e atacar
- Cada personagem realiza essas ações de maneira diferente
- Para este exercício, basta que cada método retorne uma mensagem indicando o que o personagem está fazendo

- **Implementação em Java**

- Crie um aplicativo Java (classe com método `main`) e instancie objetos de cada classe de personagem
- Chame os métodos `mover` e `atacar` para cada objeto e imprima as mensagens retornadas

## Aplicativo para desenho vetorial 2D (geometria plana)

- Modele classes para representar as seguintes formas geométricas 2D:
  - Linha
  - Círculo
  - Retângulo
- Cada forma deve ter atributos que a caracterizam, como: cor da linha, coordenadas no plano, raio, etc
- Cada forma deve ter um método *desenhar*, que retorna uma *String* com os valores dos atributos do objeto e uma mensagem indicando o tipo da forma
- Formas geométricas que possuem área devem implementar também os métodos `calcularArea` e `calcularPerimetro`
- Crie um aplicativo Java (classe com método `main`) que instancia objetos de cada classe e utiliza seus métodos